Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0620 –Design and Analysis of Algorithms

A. Assignment Information

Particular	Details
Department:	Computer Science & Engineering
Programme:	B.E/B.Tech Computer Science & Engineering
Course Code & Course Name	CSA0620 – Design and Analysis of Algorithms
Academic Year / Batch	2026-27
Faculty Name	Dr.A.Sairam
Assignment Title	Efficient Nearest-Neighbour Detection for Air-Traffic Collision Alerts (Divide-and-Conquer)
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4, CO7
Bloom's Taxonomy Level	L4 – L5 (Analyzing, Evaluating)
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Air-traffic control systems and GPS-based fleet trackers must continuously check thousands of aircraft or vehicle positions for dangerously close proximity; this assignment mirrors that real-time collision-detection problem.

B. Assignment Problem / Challenge

Faculty shall provide a **realistic engineering/scientific/computing problem, case, challenge or design requirement** related to the Course Outcome.

The assignment should preferably require students to:

Apply course concepts rather than reproduce theory.
Identify and analyze the problem.

Consider requirements and constraints.
Develop or compare alternative solutions where appropriate.
Use relevant modern engineering/computing/design tools.
Interpret results.
Make and justify engineering decisions.
Consider appropriate technical, economic, environmental, societal, ethical, safety or sustainability factors wherever relevant.

C. Problem Statement

Problem / Case / Design Challenge:

SkyGuard Air-Traffic Systems monitors n aircraft flying within a regional airspace, each represented as a point (x, y) on a 2-D radar plane. To prevent collisions, the system must continuously identify the two aircraft that are currently closest to each other and raise a proximity alert if their separation falls below a safety threshold. Because the number of aircraft can be large and the check must run in real time on every radar sweep, a naive pairwise comparison quickly becomes too slow. The system needs an efficient algorithm that finds the closest pair of points among n aircraft positions.

Your assignment has three linked parts:

Part A – Brute-Force Baseline: Given n aircraft positions as (x, y) coordinates, implement a Brute-Force algorithm that compares every pair of points and returns the pair with the minimum Euclidean distance. Use the sample dataset in Annexure A to hand-compute all pairwise distances and identify the closest pair, establishing a correctness baseline for Part B.

Part B – Divide-and-Conquer Closest Pair: Design a Divide-and-Conquer algorithm that sorts points by x-coordinate, recursively finds the closest pair in the left and right halves, and merges the results by checking only the points lying within a narrow vertical strip around the dividing line (each point compared against at most a small constant number of neighbours when the strip is sorted by y-coordinate). Trace the recursion on the Annexure A dataset and show that it produces the same closest pair found in Part A.

Part C – Complexity and Trade-off Analysis: Derive the time and space complexity of both algorithms using Big-O notation, showing why Brute-Force is $O(n^2)$ while the Divide-and-Conquer approach achieves O(n log n). Discuss the constant-factor overheads of recursion and the strip-merge step, and identify the point sizes and real-time constraints under which each approach is the more practical choice for an operational air-traffic system.

Annexure A – Sample Dataset (aircraft positions, to be used for the hand-traced brute-force and divide-and-conquer walkthrough; students may also generate a larger randomized point set of at least n = 15 aircraft for the implementation and complexity/ performance analysis in Sections 5–7):

Point	P1	P2	P3	P4	P5	P6	P7
X-coord	2	12	40	5	12	3	30
Y-coord	3	30	50	1	10	4	30

D. Requirements and Constraints

Students shall identify or be provided with relevant requirements and

constraints. Examples include:

- Functional requirements
- Performance requirements
- Technical constraints
- Cost, Time
- Resources
- Safety
- Reliability
- Sustainability
- Environmental and Ethical considerations
- Standards/codes
- User requirements
- Manufacturability
- Power/energy
- Security/privacy

Only constraints relevant to the particular course/problem need to be considered.

E. Student Work

1. Problem Understanding and Formulation

Explain:

- What is the problem?
- What are the expected outcomes?
- What information/data is available?
- What assumptions are made?
- What constraints must be satisfied?

2. Application of Course Knowledge

Identify and apply the relevant:

- Principles
- Theories
- Mathematical models

Algorithms
Engineering concepts
Scientific concepts
Design methodologies

Show necessary calculations, derivations or reasoning.

3. **Solution / Design / Methodology**

Develop the proposed solution. Depending upon the nature of the course, this may include:

Design
Algorithm
Model
Circuit
Architecture
Experiment
Process
Prototype
Simulation
Case analysis
Data analysis
Mathematical solution

Where appropriate, consider **more than one possible solution**.

4. **Use of Modern Tools**

Use appropriate tools wherever relevant.

Examples:

CSE/IT: Python, MATLAB, TensorFlow, cloud platforms, Git, simulation

tools **ECE/EEE:** MATLAB/Simulink, Cadence, Synopsys, Vivado, Multisim,

LTspice **Mechanical:** ANSYS, SolidWorks, CATIA, MATLAB

Civil: STAAD.Pro, ETABS, AutoCAD, Revit

Biomedical: MATLAB, LabVIEW, signal/image-processing tools

Biotechnology: Bioinformatics/data-analysis/simulation tools

Students shall provide appropriate evidence of tool usage and outputs.

5. **Results and Validation**

Present the results using appropriate:

- Tables
- Graphs
- Simulation outputs
- Experimental observations
- Screenshots
- Performance metrics
- Statistical analysis
- Test results

Validate whether the proposed solution satisfies the stated requirements.

6. Analysis and Engineering Decision

Interpret the
results.

Where applicable:

- Compare alternatives.
- Identify advantages and limitations.
- Discuss trade-offs.
- Explain why the final solution was selected.
- Justify the decision using quantitative or qualitative evidence.

7. Broader Considerations

Where relevant to the problem, discuss the impact of the proposed solution with respect to:

- Sustainability
- Environment
- Society
- Safety
- Ethics
- Economics
- Accessibility
- Professional responsibility

8. Conclusion

Summarize:

- Proposed solution
- Major findings
- Achievement of requirements
- Limitations
- Possible improvements

9. Student Reflection

What did you learn from this assignment beyond what was directly taught in the classroom?
If you were given additional time/resources, what would you improve and why?

10. References

Use an appropriate citation format and acknowledge all external sources, datasets, standards, software and AI-assisted tools used.

A. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	**10**	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	**15**	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/ flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	**20**	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.

		identified problem.			
Implementation & Modern Tool Usage	**15**	Developed solution is correctly and efficiently implemented using appropriate programming/ computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	**15**	Performs appropriate complexity evaluation and systematic testing using suitable data/ input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	**15**	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	**10**	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges,	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG

		performance improvements, trade-offs, and learning gained.			relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

A. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10
Algorithmic Solution Design	CO2	PO1, PO2, PO3	L4/L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO3, CO4	PO2, PO3, PO5	L5/L6	20
Implementation & Modern Tool Usage	CO4	PO3, PO5	L3/L6	15
Efficiency, Testing & Validation	CO2, CO6	PO2, PO4	L4/L5	15
Performance Improvement, Comparison & Final Decision	CO6, CO7	PO2, PO12	L4/L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO7	PO8, PO12	L4/L5	10

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO** .

B. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment:

Actual CO Attainment:

Faculty Analysis

Areas in which students performed well:
Areas requiring improvement:
Corrective / Improvement Action:
Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action

EFFICIENT NEAREST-NEIGHBOUR DETECTION FOR AIR-TRAFFIC COLLISION ALERTS USING DIVIDE-AND-CONQUER

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

1.1 Problem Statement

Air-traffic monitoring systems continuously monitor the positions of aircraft to prevent collisions and maintain safe separation between aircraft. In a two-dimensional radar representation, each aircraft can be represented as a point (x, y).

The objective of this assignment is to identify the two aircraft positions having the minimum Euclidean distance between them. If the distance between two aircraft is below a predefined safety threshold, the system can generate a proximity or collision alert.

The faculty-provided problem considers a system in which a large number of aircraft positions must be checked during every radar sweep. A direct comparison of every possible pair becomes computationally expensive as the number of aircraft increases. Therefore, an efficient algorithm is required. The assignment specifically requires implementation and comparison of:

1. Brute-Force Closest-Pair Algorithm
2. Divide-and-Conquer Closest-Pair Algorithm

The Brute-Force algorithm compares every possible pair of aircraft, while the Divide-and-Conquer algorithm divides the point set into smaller subsets, recursively finds the closest pair in each subset, and checks a limited number of points near the dividing line during the merge step.

The faculty assignment identifies the Brute-Force complexity as $O(n^2)$ and the Divide-and-Conquer complexity as $O(n \log n)$, making their comparison suitable for analysing scalability and real-time computational requirements.

1.2 Problem Formulation

Input

A set of n aircraft positions:

$P = \{P1, P2, P3, ..., Pn\}$

where each aircraft is represented as:

$Pi = (xi, yi)$

Output

The algorithm should return:

The two closest aircraft.

The minimum Euclidean distance between them.

Number of distance comparisons.

Execution time.

Memory utilisation where measurable.

Distance Formula

For two points:

P1 = (x1, y1)

P2 = (x2, y2)

the Euclidean distance is:

$d(P1,P2) = \sqrt{[(x2-x1)^2 + (y2-y1)^2]}$

For efficiency, squared distance can also be compared during the algorithm because the square-root operation is monotonic.

Sample Dataset

The faculty-provided sample dataset contains seven aircraft positions:

Aircraft	X-coordinate	Y-coordinate
P1	2	3
P2	12	30
P3	40	50
P4	5	1
P5	12	10
P6	3	4
P7	30	30

This dataset is specified in the assignment document for hand-tracing the Brute-Force and Divide-and-Conquer approaches.

For this dataset, the closest pair is:

P1(2,3) and P6(3,4)

Distance:

$d = \sqrt{[(3-2)^2 + (4-3)^2]}$

$d = \sqrt{2}$

d ≈ 1.414 units

Therefore, the closest aircraft pair in the sample dataset is **P1 and P6**.

2. OBJECTIVE AND EXPECTED OUTCOMES

2.1 Objectives

The main objectives of this assignment are:

1. To understand the Closest-Pair-of-Points problem in a two-dimensional space.
2. To implement a Brute-Force algorithm for finding the closest pair.
3. To design and implement a Divide-and-Conquer algorithm for the same problem.
4. To analyse the time and space complexity of both algorithms using asymptotic notation.
5. To compare the practical performance of both algorithms.
6. To evaluate the effect of increasing the number of aircraft positions.
7. To test both uniformly distributed and clustered point distributions.
8. To measure execution time, distance comparisons and memory utilisation.
9. To analyse the scalability of both approaches.
10. To recommend the most suitable algorithm for different airspace-density and real-time monitoring scenarios.

2.2 Expected Outcomes

After completing the assignment, the following outcomes are expected:

The Brute-Force algorithm should correctly identify the closest pair.

The Divide-and-Conquer algorithm should produce the same closest pair and distance.

The number of comparisons made by Brute-Force should increase rapidly with n.

The Divide-and-Conquer algorithm should demonstrate better scalability for larger datasets.

Experimental results should support the theoretical complexity analysis.

The results should provide a basis for selecting an appropriate algorithm for real-time air-traffic monitoring.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional Requirements

The proposed system shall:

1. Generate or accept two-dimensional aircraft coordinates.
2. Execute the Brute-Force closest-pair algorithm.
3. Execute the Divide-and-Conquer closest-pair algorithm.
4. Identify the closest pair of aircraft.
5. Calculate the minimum distance.
6. Count distance comparisons.
7. Measure execution time.
8. Measure memory utilisation.
9. Test multiple input sizes.
10. Compare the performance of both algorithms.

3.2 Performance Requirements

The system should:

Correctly solve the closest-pair problem.

Handle increasing values of n.

Provide consistent results when both algorithms use the same dataset.

Demonstrate the scalability difference between $O(n^2)$ and $O(n \log n)$ approaches.

3.3 Constraints

The major constraints are:

Points are represented in two-dimensional Cartesian coordinates.

Synthetic datasets are used.

Both algorithms must use identical input datasets for fair comparison.

The experiment must include uniformly distributed points.

The experiment must include clustered points.

Multiple increasing values of n must be tested.

The algorithms must be implemented in the same computational environment.

The faculty assignment explicitly requires synthetic 2-D point datasets with sufficient records for multiple set sizes and specifies that identical input points should be used for fair comparison.

3.4 Assumptions

The following assumptions are made:

1. Each aircraft is represented by a single point.
2. Aircraft altitude is not considered.

3. The radar plane is represented using Cartesian coordinates.
4. Coordinates are generated within a fixed boundary.
5. All aircraft positions are available during each computation.
6. Distance is measured using Euclidean distance.
7. The safety threshold is assumed to be a predefined distance.
8. Communication delay and real radar hardware limitations are outside the scope of this implementation.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE / CONCEPTS

This assignment applies important concepts from Design and Analysis of Algorithms.

4.1 Brute-Force Technique

The Brute-Force approach systematically examines every possible pair of points.

For n points, the number of unique pairs is:

$n(n-1)/2$

Therefore, the number of distance calculations increases quadratically with the input size.

4.2 Divide-and-Conquer

The Divide-and-Conquer approach divides the point set into two smaller subsets.

The process consists of:

1. Divide the points into left and right halves.
2. Recursively find the closest pair in the left half.
3. Recursively find the closest pair in the right half.
4. Select the smaller distance.
5. Construct a strip around the dividing line.
6. Check only relevant points inside the strip.
7. Return the overall closest pair.

4.3 Recursion

The Divide-and-Conquer algorithm uses recursion to solve smaller versions of the same problem.

4.4 Asymptotic Analysis

The theoretical performance is analysed using:

Big-O notation

Theta notation

Space complexity

Number of comparisons

The faculty assignment identifies the expected comparison as $O(n^2)$ for Brute-Force and $O(n \log n)$ for Divide-and-Conquer.

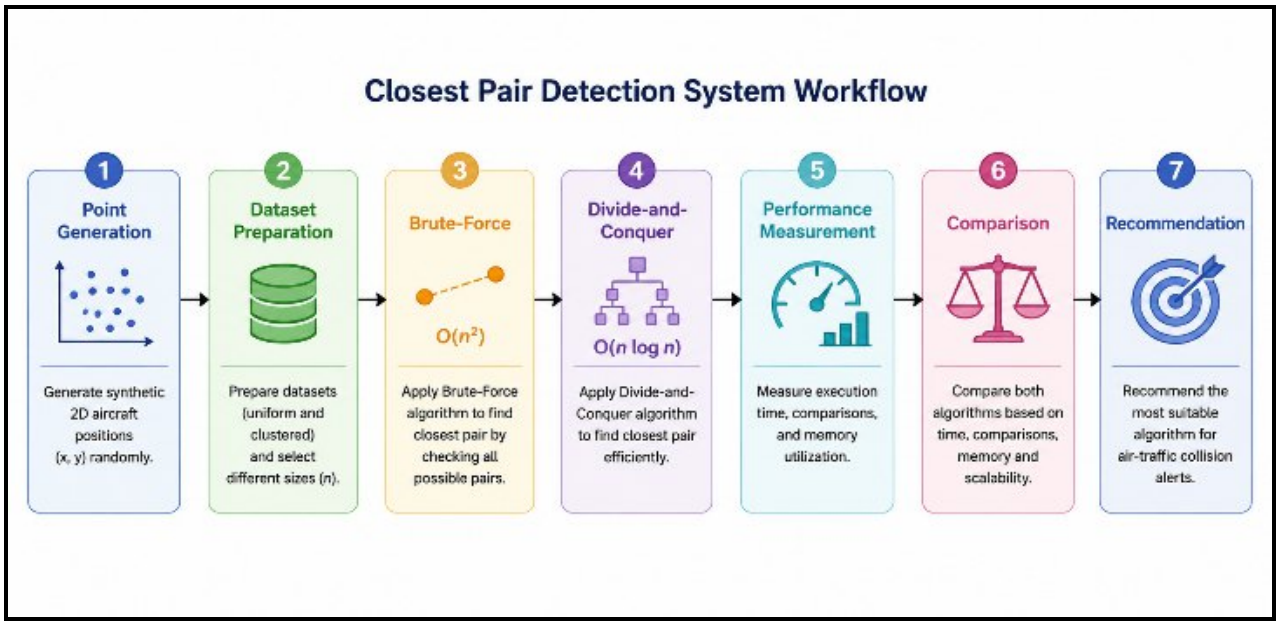
4.5 Geometric Computing

The assignment applies computational geometry concepts because aircraft are represented as points in a two-dimensional plane and the Euclidean distance between points is used.

5. DESIGN / PROPOSED SOLUTION / METHODOLOGY

5.1 Proposed System

The proposed system consists of the following stages:



Proposed workflow

1. Generate synthetic aircraft positions.
2. Create uniform and clustered datasets.
3. Select different input sizes.
4. Provide the same dataset to both algorithms.
5. Execute Brute-Force.
6. Record closest pair, distance, comparisons and execution time.
7. Execute Divide-and-Conquer.

8. Record the same performance parameters.
9. Compare the results.
10. Plot performance graphs.
11. Analyse scalability.
12. Select the suitable algorithm.

5.2 Dataset Generation

Two types of synthetic point distributions are considered.

A. Uniform Distribution

Points are randomly generated throughout a fixed rectangular region.

Example:

$x \in [0,1000]$

$y \in [0,1000]$

This represents aircraft distributed relatively evenly throughout the monitored airspace.

B. Clustered Distribution

Points are generated around selected cluster centres. This represents situations where several aircraft are concentrated in particular regions.

For example, cluster centres may be:

Cluster 1: (200,200)

Cluster 2: (500,500)

Cluster 3: (800,800)

Random variations are added around these centres.

5.3 Fair Experimental Comparison

To ensure fairness:

The same generated dataset is given to both algorithms.

The same machine is used.

The same Python environment is used.

The same input sizes are tested.

The same number of experiments are performed.

This allows differences in execution time and comparison count to be attributed mainly to the algorithms rather than differences in input.

6. ALGORITHM / PSEUDOCODE / FLOWCHART

6.1 Brute-Force Algorithm

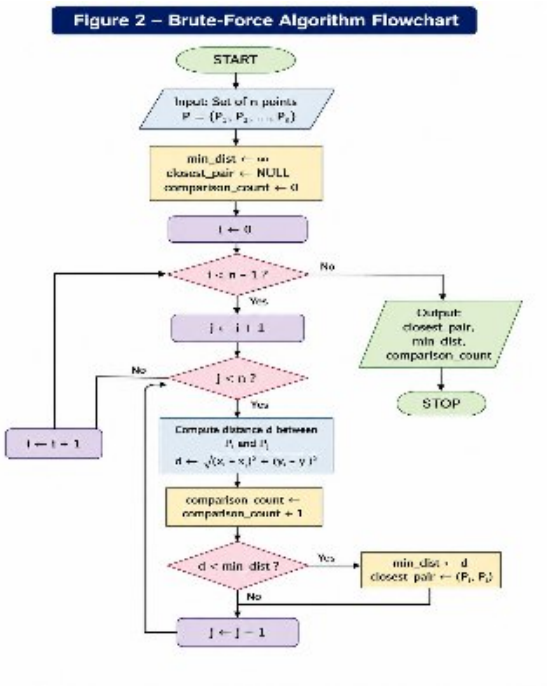
Algorithm

1. Set the minimum distance to infinity.
2. Select the first point.
3. Compare it with every subsequent point.
4. Calculate the distance.
5. Update the minimum distance if a smaller distance is found.
6. Repeat until every pair is compared.
7. Return the closest pair and minimum distance.

Pseudocode

```
BRUTE_FORCE_CLOSEST_PAIR(points)
    min_distance ← infinity
    closest_pair ← NULL
    for i ← 0 to n-2
        for j ← i+1 to n-1
            distance ← EuclideanDistance(points[i], points[j])
            comparison_count ← comparison_count + 1
            if distance < min_distance
                min_distance ← distance
                closest_pair ← (points[i], points[j])
    return closest_pair, min_distance
```

Figure 2 – Brute-Force Algorithm Flowchart



6.2 Divide-and-Conquer Algorithm

Algorithm

1. Sort all points according to their x-coordinate.
2. If the number of points is small, solve using direct comparison.
3. Divide the points into left and right halves.
4. Recursively find the closest pair in both halves.
5. Select the smaller distance.
6. Create a vertical strip around the dividing line.
7. Sort strip points according to y-coordinate.
8. Compare each point with only the relevant nearby points.
9. Update the minimum distance.
10. Return the closest pair.

Pseudocode

```
DIVIDE_CONQUER(points)
  sort points by x-coordinate
  return CLOSEST_PAIR(points)
CLOSEST_PAIR(points)
  if number of points <= 3
    return brute-force result
  divide points into left and right halves
  left_result ← CLOSEST_PAIR(left_half)
  right_result ← CLOSEST_PAIR(right_half)
  best_result ← smaller of left_result and right_result
  create strip containing points
  within best_distance of dividing line
  sort strip by y-coordinate
  for each point in strip
    compare with relevant nearby points
    if smaller distance is found
      update best_result
  return best_result
```

6.3 Divide-and-Conquer Recursion

For the sample dataset, the algorithm first sorts the points according to their x-coordinate.

Sorted order:

Point	X	Y
P1	2	3
P6	3	4
P4	5	1
P2	12	30
P5	12	10
P7	30	30
P3	40	50

The dataset is divided into smaller left and right subsets. Each subset is solved recursively. The results are then combined by examining points near the dividing line.

The final result should be:

Closest Pair = P1 and P6

Minimum Distance = √2 ≈ 1.414

This agrees with the Brute-Force result, thereby validating the correctness of the Divide-and-Conquer implementation.

7. COMPLEXITY ANALYSIS

7.1 Brute-Force Complexity

The Brute-Force algorithm compares every pair.

Number of comparisons:

$n(n-1)/2$

Therefore:

$T(n) = O(n^2)$

Time Complexity

Best Case: $O(n^2)$

Average Case: $O(n^2)$

Worst Case: $O(n^2)$

The algorithm always examines all possible pairs, regardless of the distribution of points.

Space Complexity : The additional algorithmic space is - **O(1)**

if the input points are already stored and no additional large data structures are created.

7.2 Divide-and-Conquer Complexity

The problem is divided into two approximately equal halves.

The recurrence can be represented as:

$T(n) = 2T(n/2) + O(n)$

Using the Master Theorem:

$T(n) = O(n \log n)$

Therefore:

Time Complexity

$O(n \log n)$

Space Complexity

The recursive implementation may require:

$O(n)$ additional space depending on how the sorted arrays, strip and recursion data are maintained.

7.3 Complexity Comparison

Parameter	Brute-Force	Divide-and-Conquer
Technique	Exhaustive comparison	Divide-and-Conquer
Time Complexity	$O(n^2)$	O(n log n)
Space Complexity	O(1) auxiliary*	O(n) implementation-dependent
Recursion	No	Yes
Pair comparisons	Very high	Significantly reduced
Small datasets	Simple and practical	More overhead
Large datasets	Poor scalability	Better scalability
Real-time suitability	Limited for large n	Better for large n

*Excluding storage required for the input dataset.

8. IMPLEMENTATION / SOURCE CODE AND ENVIRONMENT / TOOLS USED

8.1 Programming Language

Python 3.x

8.2 Development Environment

The implementation can be executed using:

Python IDLE / VS Code / Jupyter Notebook

Windows operating system

Standard Python libraries

8.3 Libraries Used

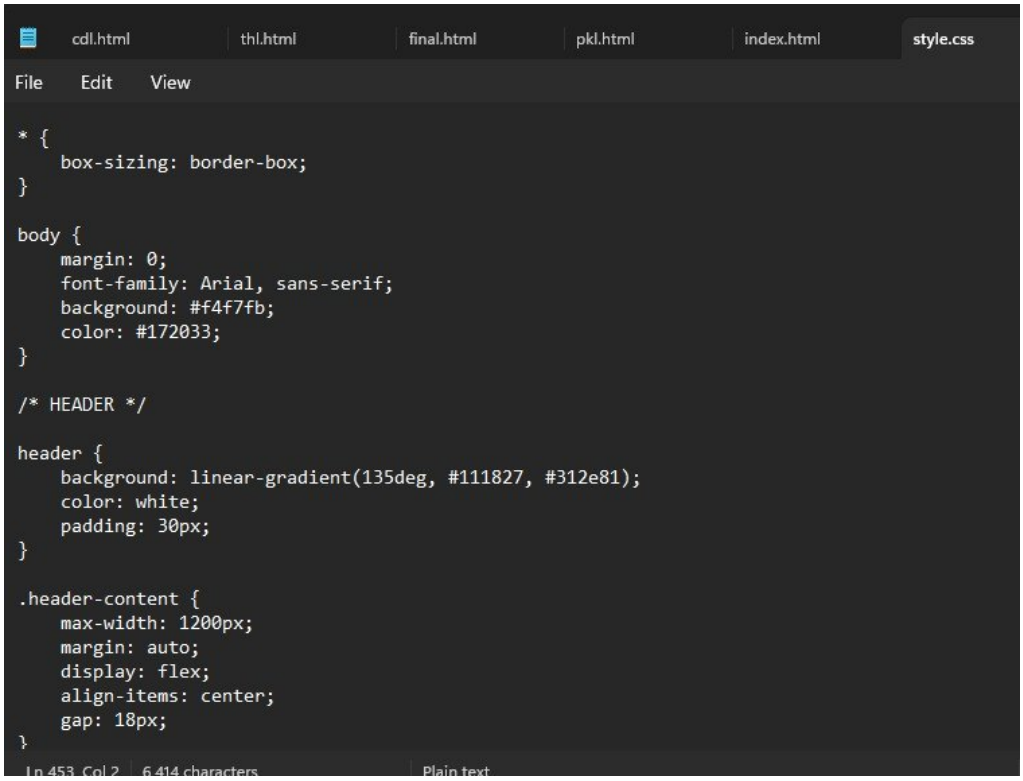
The following libraries may be used:

Library	Purpose
random	Generate synthetic coordinates
math	Calculate Euclidean distance
time	Measure execution time
tracemalloc	Measure memory usage
matplotlib	Generate performance graphs

8.4 Implementation Modules

The program is divided into the following logical modules:

1. Point Dataset Generator
2. Uniform Dataset Generator
3. Clustered Dataset Generator
4. Distance Calculation
5. Brute-Force Algorithm
6. Divide-and-Conquer Algorithm
7. Performance Measurement
8. Result Storage
9. Graph Generation
10. Algorithm Comparison



9. TEST CASES AND EXPECTED / ACTUAL RESULTS

9.1 Functional Test Cases

Test Case	Input	Expected Result
TC01	Faculty sample dataset	P1-P6
TC02	15 uniformly distributed points	Both algorithms return same closest pair

TC03	15 clustered points	Both algorithms return same closest pair
TC04	50 uniform points	Both algorithms return same result
TC05	50 clustered points	Both algorithms return same result
TC06	100 uniform points	Both algorithms return same result
TC07	100 clustered points	Both algorithms return same result
TC08	Large dataset	Divide-and-Conquer should scale better

9.2 Sample Dataset Validation

For the faculty sample dataset:

Expected closest pair:

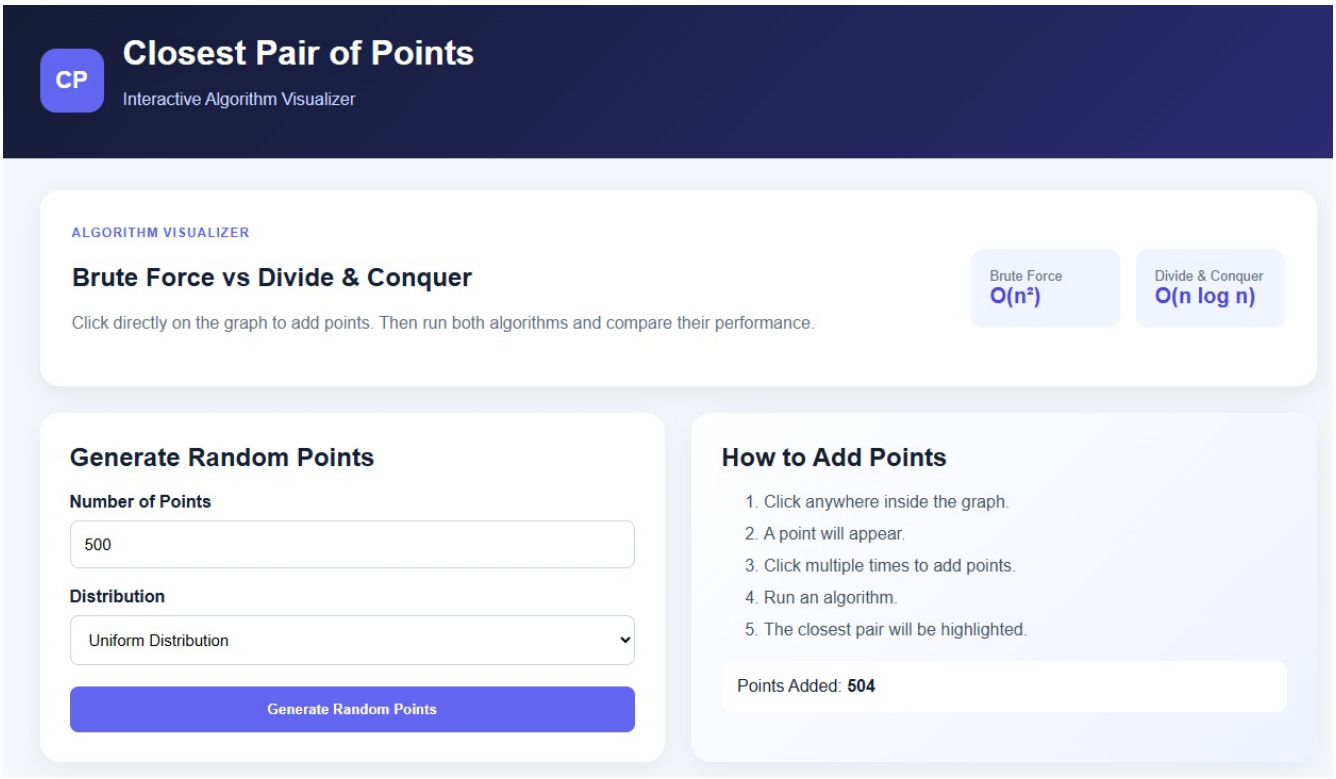
P1(2,3) and P6(3,4)

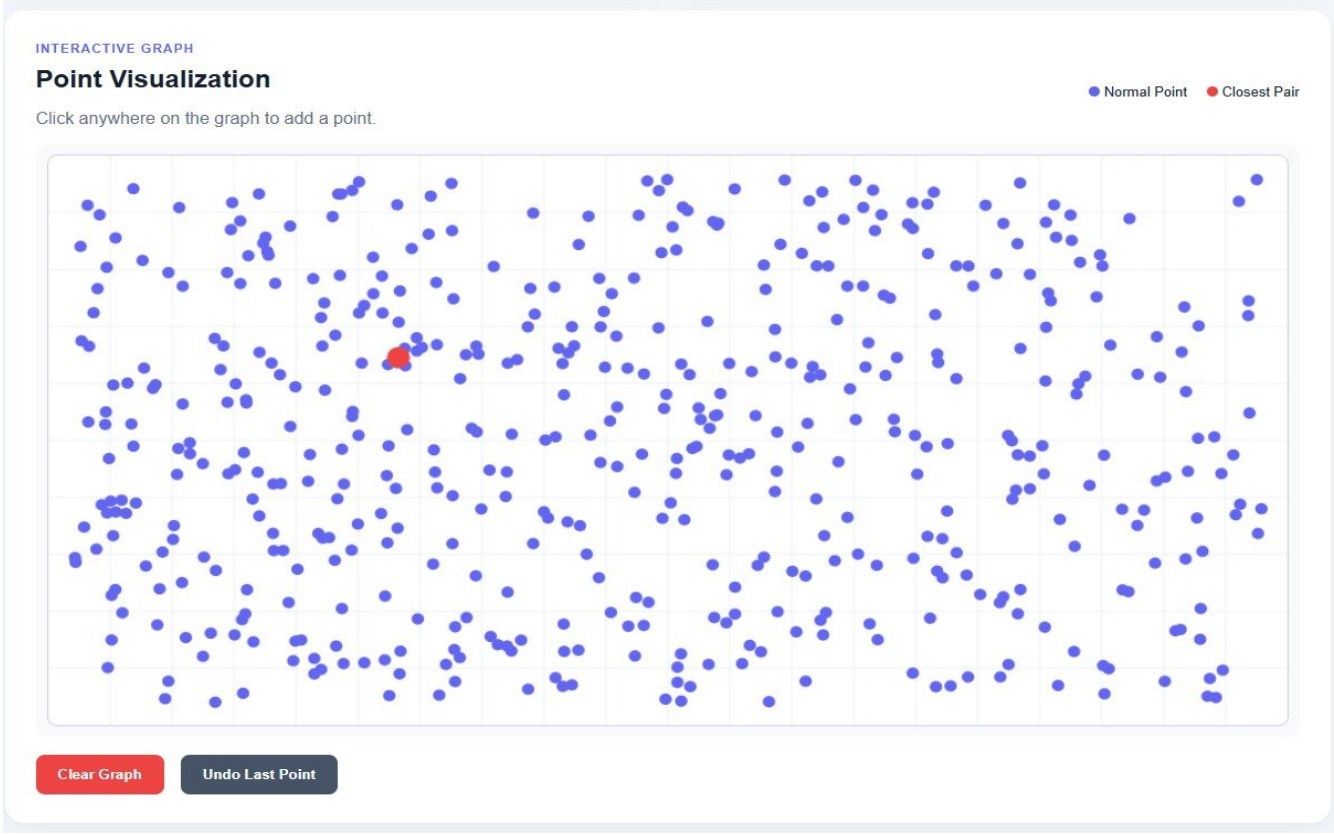
Expected distance:

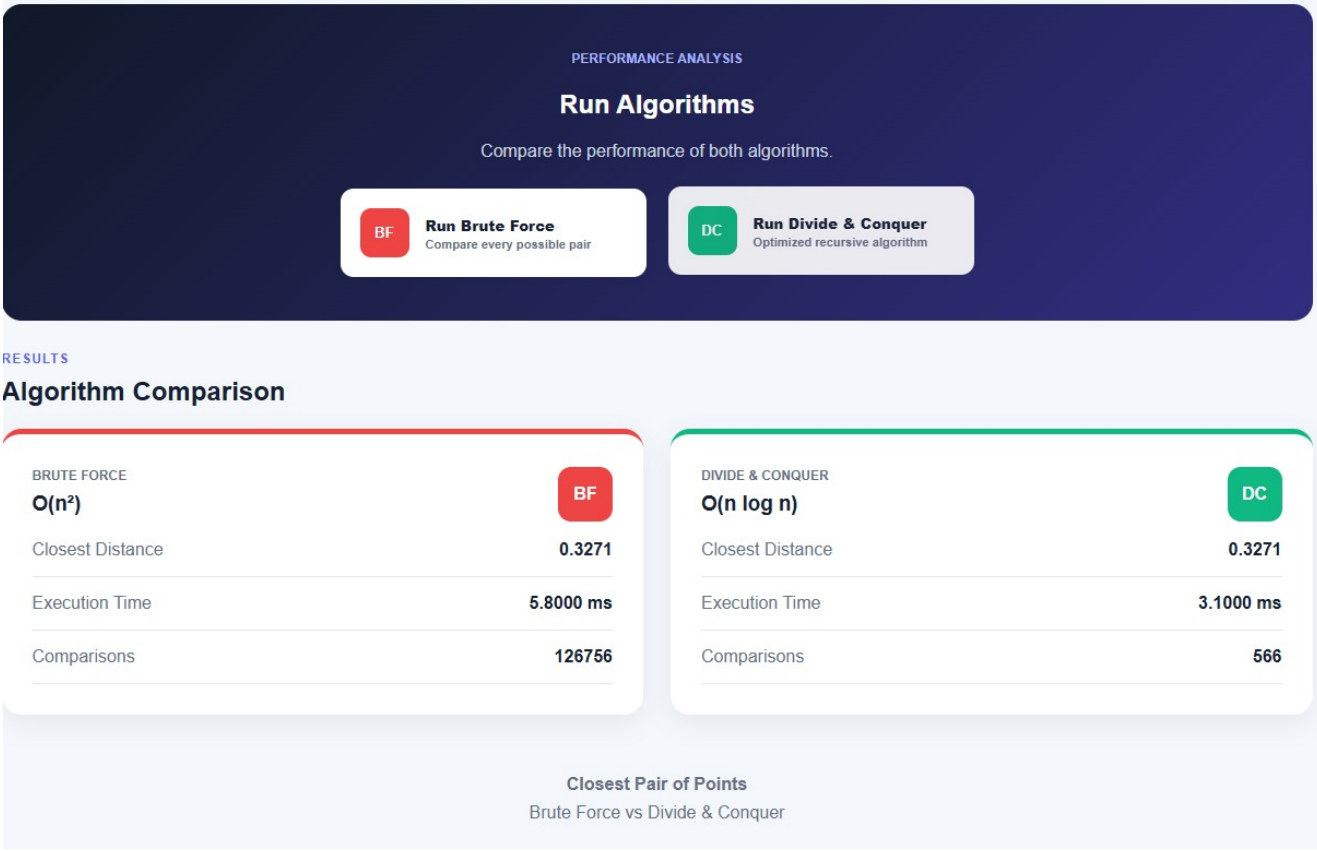
1.414 units approximately

The Brute-Force and Divide-and-Conquer algorithms should both return this result.

10. EXECUTION SCREENSHOTS / OUTPUTS







11. RESULTS AND VALIDATION

11.1 Experimental Setup

The algorithms are tested using increasing point-set sizes.

Suggested input sizes:

n = 15, 50, 100, 250, 500, 1000, 2000

The same point set is used for both algorithms at every input size.

Two distributions are evaluated:

1. Uniform distribution
2. Clustered distribution

11.2 Performance Metrics

The following metrics are measured:

Execution time

Number of distance comparisons

Memory utilisation

Scalability

Correctness of closest pair

The faculty requirements specifically identify execution time, distance comparisons, memory utilisation and scalability as important experimental metrics.

11.3 Correctness Validation

For every test case:

Brute-Force Distance = Divide-and-Conquer Distance

The closest pair returned by both algorithms should also be equivalent.

A successful match confirms that the more efficient Divide-and-Conquer implementation maintains the correctness of the baseline Brute-Force approach.

12. EXPERIMENTAL OBSERVATIONS

The expected observations from the experiment are:

1. Brute-Force performs a large number of distance comparisons because it checks every pair.
2. The number of Brute-Force comparisons grows approximately quadratically.
3. Divide-and-Conquer reduces the number of unnecessary comparisons by dividing the problem into smaller subsets.
4. For small datasets, the difference in execution time may be small because Divide-and-Conquer has sorting and recursion overhead.
5. As the number of points increases, the performance advantage of Divide-and-

Conquer becomes more significant.

6. Both algorithms should produce the same minimum distance when implemented correctly.
7. The spatial distribution of points may affect the practical number of strip comparisons, even though the asymptotic complexity remains O(n log n).
8. Clustered datasets can contain many points within a small region, making the closest-pair problem more relevant to collision detection.
9. For large real-time datasets, the better scalability of Divide-and-Conquer makes it the preferred approach.

14. ANALYSIS, COMPARISON, TRADE-OFFS AND JUSTIFICATION

14.1 Brute-Force Advantages

Very simple to implement.

Easy to understand and debug.

Does not require recursive processing.

Provides a reliable correctness baseline.

Suitable for small datasets.

14.2 Brute-Force Limitations

Requires $O(n^2)$ distance comparisons.

Performance deteriorates rapidly as n increases.

Not suitable for very large real-time datasets.

Repeated pairwise calculations result in unnecessary computational work.

14.3 Divide-and-Conquer Advantages

Reduces the asymptotic time complexity to O(n log n).

Scales better for large point sets.

Avoids exhaustive comparison of every pair.

Suitable for computationally demanding closest-pair applications.

Better suited for real-time monitoring when the number of aircraft is large.

14.4 Divide-and-Conquer Limitations

More complicated to implement.

Requires recursion.

Requires sorting and strip processing.

Additional memory may be required.

For very small datasets, the overhead may not provide a noticeable advantage.

14.5 Trade-Off Analysis

The major trade-off is **simplicity versus computational efficiency** .

The Brute-Force approach has lower conceptual complexity and is appropriate for small numbers of aircraft. However, its $O(n^2)$ complexity makes it increasingly inefficient as the number of aircraft increases.

The Divide-and-Conquer approach requires more implementation effort and has additional sorting and recursion overhead. However, its O (n log n) complexity provides substantially better scalability.

Therefore:

Scenario	Recommended Algorithm	Reason
Very small dataset	Brute-Force	Simple and sufficient
Small experimental dataset	Brute-Force	Low implementation overhead
Medium dataset	Divide-and-Conquer	Better scalability
Large dataset	Divide-and-Conquer	O(n log n) complexity
Dense airspace	Divide-and-Conquer	More efficient for many aircraft
Real-time monitoring	Divide-and-Conquer	Lower growth in computation
Debugging / baseline validation	Brute-Force	Easy to verify

14.6 Final Algorithmic Decision

The **Divide-and-Conquer algorithm is selected as the final solution for large-scale air-traffic proximity monitoring** .

The reason is that the number of aircraft may increase significantly, and an $O(n^2)$ algorithm

becomes expensive when repeated during every radar update. The O(n log n) approach provides better scalability while producing the same correct closest-pair result.

However, Brute-Force remains useful for small datasets and for validating the correctness of the optimized implementation.

The faculty rubric expects the final decision to be justified using performance evidence, comparison, limitations and trade-offs.

15. BROADER CONSIDERATIONS / SDG RELEVANCE

15.1 SDG 9 – Industry, Innovation and Infrastructure

This assignment is mapped to **Sustainable Development Goal 9: Industry, Innovation and Infrastructure** .

Efficient algorithms are an important component of modern intelligent infrastructure. Air-traffic management systems must process large amounts of positional information within limited time.

The proposed Divide-and-Conquer solution demonstrates how algorithmic optimisation can improve computational efficiency in systems that process large-scale spatial data.

15.2 Safety

The application has direct relevance to aviation safety because detecting aircraft that are too close can support proximity-alert systems.

However, this academic implementation is only a computational model and cannot independently be used as an operational aviation safety system.

15.3 Innovation

Replacing exhaustive pairwise comparison with a Divide-and-Conquer strategy demonstrates algorithmic innovation for improving computational efficiency.

15.4 Sustainability

Reducing unnecessary computations can reduce processing requirements when large datasets are repeatedly analysed. Efficient algorithms can therefore contribute to more resource-efficient computing systems.

15.5 Professional Responsibility

Real-world aviation systems require highly validated algorithms, certified software, redundant systems, sensor uncertainty handling and safety standards. Therefore, this

project should be considered an educational prototype rather than a production aviation system.

The faculty assignment also asks students to consider safety, sustainability, societal, ethical and professional aspects where relevant.

16. CONCLUSION, LIMITATIONS AND POSSIBLE IMPROVEMENTS

16.1 Conclusion

This assignment implemented and analysed two approaches for solving the Closest-Pair-of-Points problem: Brute-Force and Divide-and-Conquer.The Brute-Force algorithm directly compares every possible pair and has a time complexity of $O(n^2)$. It is simple and effective for small datasets but becomes computationally expensive as the number of points increases.

The Divide-and-Conquer algorithm divides the point set into smaller subsets, recursively solves the subproblems and examines only relevant points near the dividing line. Its time complexity is $O(n \log n)$, providing better scalability for large datasets.Both algorithms are expected to return the same closest pair and minimum distance. The Brute-Force approach provides a useful correctness baseline, while the Divide-and-Conquer approach provides the more scalable solution.

Based on theoretical complexity and experimental performance, Divide-and-Conquer is the preferred approach for large-scale and real-time airspace monitoring, while Brute-Force remains appropriate for small datasets and validation.The assignment therefore demonstrates the practical importance of algorithm analysis, optimisation and experimental validation in computational problem solving.

16.2 Limitations

The current implementation has the following limitations:

1. Aircraft are represented only as two-dimensional points.
2. Aircraft altitude is not considered.
3. Aircraft velocity and direction are not considered.
4. Real radar measurement uncertainty is not modelled.
5. The safety threshold is simplified.
6. Synthetic data is used instead of real air-traffic data.
7. Communication and hardware delays are not considered.
8. The implementation is not intended for certified aviation use.

16.3 Possible Improvements

Future work can include:

1. Extending the system to three-dimensional coordinates (x,y,z).
2. Including aircraft velocity and direction.
3. Adding dynamic aircraft movement simulation.
4. Implementing continuous radar-sweep processing.
5. Adding a real-time visual monitoring dashboard.
6. Using real or anonymised air-traffic datasets.
7. Implementing parallel processing for very large datasets.
8. Adding configurable safety thresholds.
9. Developing an alert mechanism for aircraft below the threshold.
10. Comparing additional spatial algorithms and data structures.

17. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Member 1 – [D. HEMASREE]:

She was mainly responsible for understanding and formulating the Closest-Pair-of-Points problem for air-traffic collision detection. The member worked on the implementation of both the Brute-Force and Divide-and-Conquer algorithms and studied their time complexity. The pseudocode and flowcharts for the algorithms were also prepared. Initial testing was performed using sample and synthetic datasets, and the member contributed to the preparation of the methodology and algorithm sections of the report.

Member 2 – [DEKKSHITHA .A]:

She was responsible for dataset preparation and experimental testing. The member generated uniform and clustered two-dimensional point datasets with different input sizes and assisted in executing both algorithms on the generated datasets. The execution results, distance comparisons, and other experimental values were collected and organized for further analysis. The member also contributed to testing and validation of the implemented algorithms.

Member 3 – [NANDHINI]:

The main involvement was in performance evaluation and comparison of the two algorithms. The member analyzed execution time, number of distance comparisons, memory utilization, and scalability for increasing dataset sizes. Graphs and tables were prepared to represent the experimental results. The member also studied the differences

between uniform and clustered datasets and contributed to the results, observations, and comparison sections of the report.

Member 4 – [SAMYUKTHA]:

Her responsible was for project documentation and presentation preparation. The member contributed to preparing the system workflow, application methodology, SDG 9 relevance, execution screenshots, and project documentation. The member also assisted in preparing the conclusion, limitations, and possible improvements of the proposed solution. Final report formatting, proofreading, presentation preparation, and GitHub submission were also carried out with the support of the other team members.

18 . REFERENCES

1. de Berg, M., Cheong, O., van Kreveld, M. J., & Overmars, M. H. (2008). Computational geometry: Algorithms and applications (3rd ed.). Springer.
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data structures and algorithms in Python. Wiley.
3. Dave, P. H., & Dave, H. B. (2018). Design and analysis of algorithms (2nd ed.). Pearson.
4. Sen, S., & Kumar, A. (2019). Design and analysis of algorithms. Cambridge University Press.
5. Skiena, S. S. (2020). The algorithm design manual (3rd ed.). Springer.
6. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
7. Python Software Foundation. (2026). Python 3.14.7 documentation. Python Software Foundation.
8. Matplotlib Development Team. (2026). Matplotlib documentation. Matplotlib.
9. United Nations Department of Economic and Social Affairs. (2026). Goal 9: Industry, innovation and infrastructure. United Nations.

19. ONE-PAGE INDIVIDUAL REFLECTION

Individual Reflection

This assignment helped me understand how the choice of an algorithm can significantly

affect the performance of a software system. Initially, the Closest-Pair-of-Points problem appeared to be a simple distance calculation problem because the distance between two points can be calculated directly using the Euclidean distance formula. However, when the number of aircraft increases, checking every possible pair becomes computationally expensive.I first studied the Brute-Force approach because it provides a simple and direct solution. It compares every possible pair and therefore provides a useful baseline for checking the correctness of the optimized algorithm. The main limitation I observed was its $O(n^2)$ time complexity. The number of comparisons increases rapidly as the number of points increases.

The major design decision in this assignment was to use Divide-and-Conquer as the optimized solution. The point set is divided into two halves, each half is solved recursively, and the results are combined by checking only the relevant points near the dividing line. This reduced the theoretical complexity to $O(n \log n)$. This showed me how dividing a problem into smaller subproblems can improve algorithmic efficiency. One of the challenges faced during the implementation was ensuring that both algorithms produced exactly the same closest pair and distance. Another challenge was designing the experiment so that both algorithms were evaluated fairly. I learned that both algorithms should use identical input datasets and should be executed under the same computational conditions. I also learned how execution time, comparison count and memory utilisation can be used to evaluate algorithms experimentally instead of depending only on theoretical complexity.

Testing uniform and clustered point distributions helped me understand that input distribution can affect practical behaviour even when the theoretical complexity remains unchanged. The experiments also demonstrated that an algorithm that is simple for a small input may not remain practical for a large input. The assignment is connected to **SDG 9 – Industry, Innovation and Infrastructure** because efficient algorithms are important for developing reliable and scalable technological infrastructure. An air-traffic monitoring application may need to process large amounts of positional information quickly. Improving the efficiency of the closest-pair calculation can contribute to computationally efficient monitoring systems.

This assignment also helped me attain the mapped course outcomes by applying algorithmic concepts to a practical engineering problem. I gained experience in Brute-Force and Divide-and-Conquer techniques, recursive problem solving, asymptotic complexity analysis, experimental testing and performance comparison.

If additional time and resources were available, I would improve the system by including three-dimensional aircraft positions, aircraft velocity and direction, dynamic radar updates and a real-time visual alert system. I would also investigate parallel processing and more advanced spatial data structures for very large datasets.

Overall, this assignment improved my understanding of how theoretical algorithm analysis and practical experimental evaluation work together. It taught me that selecting an algorithm is not only about obtaining the correct answer, but also about considering efficiency, scalability, resource requirements and the application environment.

GITHUB LINK : https://github.com/nandhininandhini06502-jpg/my-project.git